



МИНОБРНАУКИ РОССИИ

**федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)**

**Основная образовательная программа 09.03.02
«Информационные системы и технологии»**

**Отчет по дисциплине «Проектная деятельность»
по домашней работе на семинаре №2**

Тема: «Простой детектор без ML»

**Проверил
к.т.н. доцент**

Ковалев И.А.

подпись

**Выполнила
студентка группы ИДБ-25-06**

Якунина В.С.

подпись

Оглавление

ВВЕДЕНИЕ.....	3
ПРОГРАММА.....	3
ТЕСТИРОВАНИЕ.....	7
ЗАКЛЮЧЕНИЕ.....	13

ВВЕДЕНИЕ

Целью работы является знакомство с логикой анализа телеметрии космического аппарата и понимание ограничения правил, заданных человеком, до перехода к машинному обучению . Ссылка на [github](#) с программой.

ПРОГРАММА

Для выполнения задачи необходимо было доработать алгоритм, который анализирует данные о характеристиках спутника, анализирует и обрабатывает их. Детектор был сделан более гибким.

```
import pandas as pd
import matplotlib.pyplot as plt

def get_intervals(df, col='anomaly'):
    mask = df[col] == 1
    grp = (mask != mask.shift()).cumsum()

    intervals = (
        df[mask]
        .groupby(grp)
        .agg(
            start=('timestamp', 'first'),
            end=('timestamp', 'last'),
            rows=('timestamp', 'size')
        )
        .reset_index(drop=True)
    )
    intervals['duration_min'] = (
        (intervals['end'] - intervals['start']).dt.total_seconds() / 60
    )
    return intervals

def detect_combin(row):
    result = int(row['temperature'] > 65) + int(row['voltage'] < 23) +
    int(row['current'] > 9) + int(row['angular_velocity'] > 1.5)
    return int(result >= 2)
```

```

def detect_hard(row):
    return int(
        row['temperature'] > 65 or
        row['voltage'] < 23 or
        row['current'] > 9 or
        row['angular_velocity'] > 1.5
    )

def detect_stat(df):
    mean_t = df['temperature'].mean()
    std_t = df['temperature'].std()
    return (abs(df['temperature'] - mean_t) > 2 * std_t).astype(int)

def main():
    filename = "telemetry_dzz_sem1.csv"

    df = pd.read_csv(filename, parse_dates=['timestamp'])
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df = df.sort_values('timestamp').reset_index(drop=True)

    print('!!!!!!!!!!!!!!!!!!!! TELEMETRY (5) !!!!!!!!!!!!!!!!!!!!!')
    print(df.head())
    print()

    df['anomaly_hard'] = df.apply(detect_hard, axis=1)
    df['anomaly_stat'] = detect_stat(df)
    df['anomaly_comb'] = df.apply(detect_combin, axis=1)

    params = ['temperature', 'voltage', 'current', 'angular_velocity']
    for col in params:
        print("-----")
        print(col)
        print("-----")
        print(df.groupby('mode')[col].describe())
        print("-----")
    print()

    intervals_hard = get_intervals(df, 'anomaly_hard')
    intervals_stat = get_intervals(df, 'anomaly_stat')
    intervals_comb = get_intervals(df, 'anomaly_comb')

    print("*****")
    print("INTERVALS - hard")
    print("*****")
    print(intervals_hard)

```

```

print()
print("*****")
print("INTERVALS – statistical")
print("*****")
print(intervals_stat)
print()
print("*****")
print("INTERVALS – combined")
print("*****")
print(intervals_comb)
print()
print("*****")
print()

print("(((((((((((((((((((((")
print("DETECTOR COMPARISON")
print(f"Hard:          anomaly rows = {df['anomaly_hard'].sum()}, intervals = {len(intervals_hard)}")
print(f"Statistical:  anomaly rows = {df['anomaly_stat'].sum()}, intervals = {len(intervals_stat)}")
print(f"Combined:      anomaly rows = {df['anomaly_comb'].sum()}, intervals = {len(intervals_comb)}")
print("))))))))))))))))))")
print()

for col in params:
    plt.figure(figsize=(12, 4))
    plt.plot(df['timestamp'], df[col], label=col, color='tab:blue')

    plt.scatter(
        df.loc[df['anomaly_hard'] == 1, 'timestamp'],
        df.loc[df['anomaly_hard'] == 1, col],
        color='orange', s=25, label='hard', zorder=5
    )
    plt.scatter(
        df.loc[df['anomaly_stat'] == 1, 'timestamp'],
        df.loc[df['anomaly_stat'] == 1, col],
        color='red', s=30, label='stat', zorder=6
    )
    plt.scatter(
        df.loc[df['anomaly_comb'] == 1, 'timestamp'],
        df.loc[df['anomaly_comb'] == 1, col],
        color='purple', s=35, label='comb', zorder=7
    )

    for _, row in intervals_hard.iterrows():
        plt.axvspan(row['start'], row['end'], color='orange', alpha=0.10)

```

```

for _, row in intervals_stat.iterrows():
    plt.axvspan(row['start'], row['end'], color='red', alpha=0.10)
for _, row in intervals_comb.iterrows():
    plt.axvspan(row['start'], row['end'], color='purple', alpha=0.10)

plt.title(f'Telemetry: {col}')
plt.xlabel('timestamp')
plt.ylabel(col)

df['temperature_diff'] = df['temperature'].diff()
df['voltage_diff'] = df['voltage'].diff()
df['current_diff'] = df['current'].diff()
df['angular_velocity_diff'] = df['angular_velocity'].diff()

params = ['temperature_diff', 'voltage_diff', 'current_diff',
'angular_velocity_diff']
params_diff = [5, 0.4, 1, 0.1]
cnt = 0

for col in params:
    plt.figure(figsize=(12, 4))
    plt.plot(df['timestamp'], df[col], label=col, color='tab:blue')
    plt.scatter(
        df.loc[abs(df[col]) > params_diff[cnt], 'timestamp'],
        df.loc[abs(df[col]) > params_diff[cnt], col],
        color='green', s=25, zorder=5
    )
    plt.title(f'Telemetry: {col}')
    plt.xlabel('timestamp')
    plt.ylabel(col)
    cnt += 1

plt.show()
print()
print('xxxxxxxxxxxxxxxxxxxxx CLOSE FIGURES xxxxxxxxxxxxxxxxxxxxxxxx')
print()

if __name__ == "__main__":
    main()

```

ТЕСТИРОВАНИЕ

Была проведена проверка работы программы. Результаты проверки представлены на рисунках 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 и 11.

```
!!!!!!!!!!!!!!!!!!!!!! TELEMETRY (5) !!!!!!!!!!!!!!!!!!!!!!!
```

	timestamp	temperature	voltage	current	angular_velocity	mode
0	2026-09-07 08:00:00	23.94	28.24	1.44	0.033	standby
1	2026-09-07 08:01:00	24.11	28.13	1.48	0.022	standby
2	2026-09-07 08:02:00	24.23	28.23	1.48	0.039	standby
3	2026-09-07 08:03:00	24.79	28.22	1.41	0.013	standby
4	2026-09-07 08:04:00	24.77	28.29	1.48	0.024	standby

temperature

	count	mean	std	min	25%	50%	75%	max
mode								
attitude_control	100.0	29.054300	0.861798	26.62	28.5150	29.195	29.690	30.85
downlink	120.0	32.384000	8.914038	27.44	28.8600	29.590	31.490	65.16
imaging	160.0	38.831063	6.763547	32.50	36.2050	37.480	38.905	68.23
standby	340.0	24.170647	1.880855	20.67	22.6375	24.265	25.720	28.27

voltage

	count	mean	std	min	25%	50%	75%	max
mode								
attitude_control	100.0	27.553100	0.106455	27.31	27.48	27.55	27.6225	27.91
downlink	120.0	27.206250	1.624458	23.09	27.73	27.85	27.9725	28.16
imaging	160.0	27.434625	0.111259	27.15	27.35	27.43	27.5125	27.77
standby	340.0	28.162412	0.521321	24.00	28.10	28.22	28.3400	28.61

Рисунок 1

```

-----
current
-----
mode
count      mean      std      min      25%      50%      75%      max
attitude_control  100.0  5.659500  0.237573  5.14  5.4900  5.680  5.8075  6.16
downlink          120.0  3.893417  1.826895  2.68  3.0600  3.270  3.5425  10.30
imaging           160.0  5.219125  1.059717  4.06  4.7775  5.030  5.2800  10.32
standby           340.0  1.469471  0.113905  1.23  1.3900  1.475  1.5500  1.74
-----

-----
angular_velocity
-----
mode
count      mean      std      min      25%      50%      75%      max
attitude_control  100.0  0.428720  0.171691  0.007  0.31775  0.4215  0.55900  0.768
downlink          120.0  0.094992  0.239346  0.001  0.01875  0.0300  0.03825  1.000
imaging           160.0  0.034063  0.016505  0.000  0.02300  0.0330  0.04200  0.083
standby           340.0  0.084529  0.338378  0.000  0.01800  0.0260  0.03500  2.431
-----

*****
INTERVALS – hard
*****
start      end      rows      duration_min
0 2026-09-07 09:18:00 2026-09-07 09:21:00      4      3.0
1 2026-09-07 14:59:00 2026-09-07 14:59:00      1      0.0
2 2026-09-07 15:01:00 2026-09-07 15:06:00      6      5.0
3 2026-09-07 16:55:00 2026-09-07 17:04:00     10      9.0
4 2026-09-07 18:49:00 2026-09-07 18:56:00      8      7.0

```

Рисунок 2

```

*****
INTERVALS – statistical
*****

      start                end  rows  duration_min
0 2026-09-07 09:12:00 2026-09-07 09:21:00    10        9.0
1 2026-09-07 18:48:00 2026-09-07 18:56:00     9        8.0

*****

INTERVALS – combined
*****

      start                end  rows  duration_min
0 2026-09-07 18:55:00 2026-09-07 18:56:00     2        1.0

*****

((((((((((((((((
DETECTOR COMPARISON
Hard:      anomaly rows = 29, intervals = 5
Statistical: anomaly rows = 19, intervals = 2
Combined:   anomaly rows = 2, intervals = 1
))))))))))))))

```

Рисунок 3

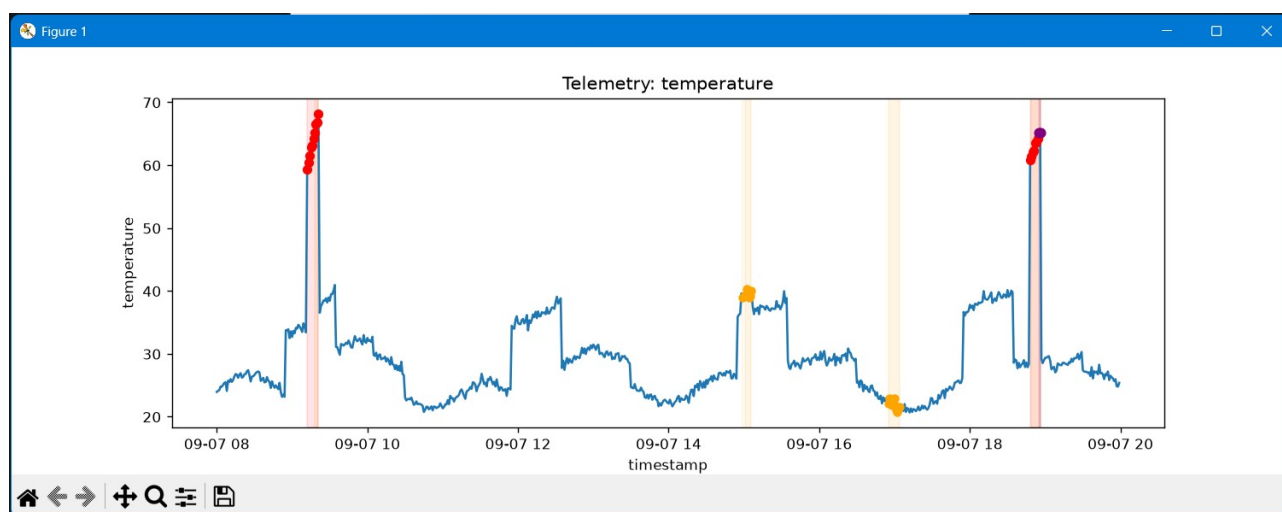


Рисунок 4

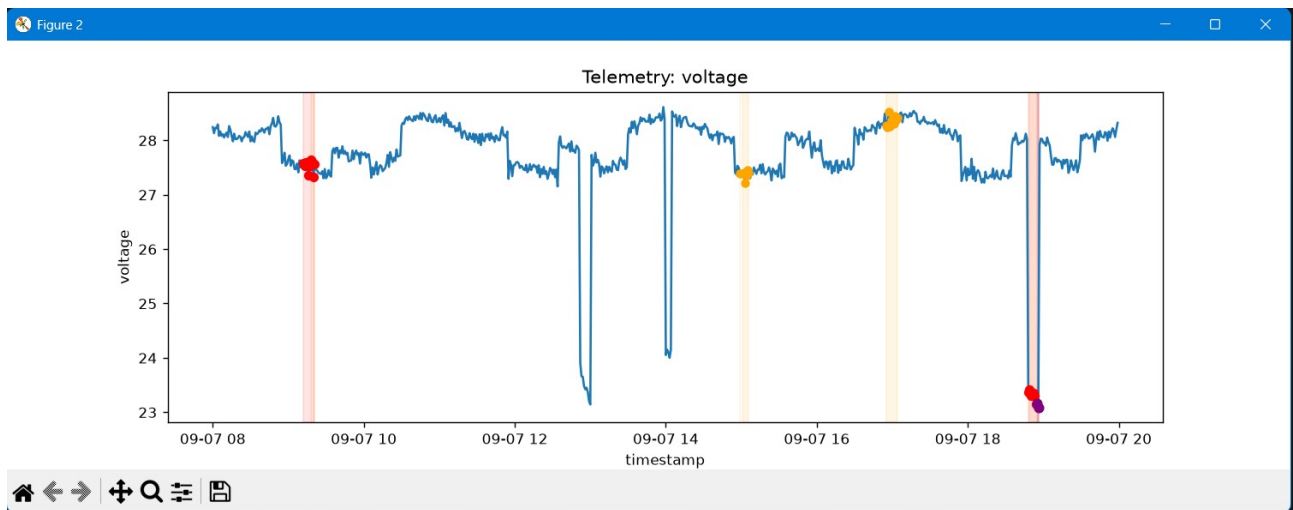


Рисунок 5

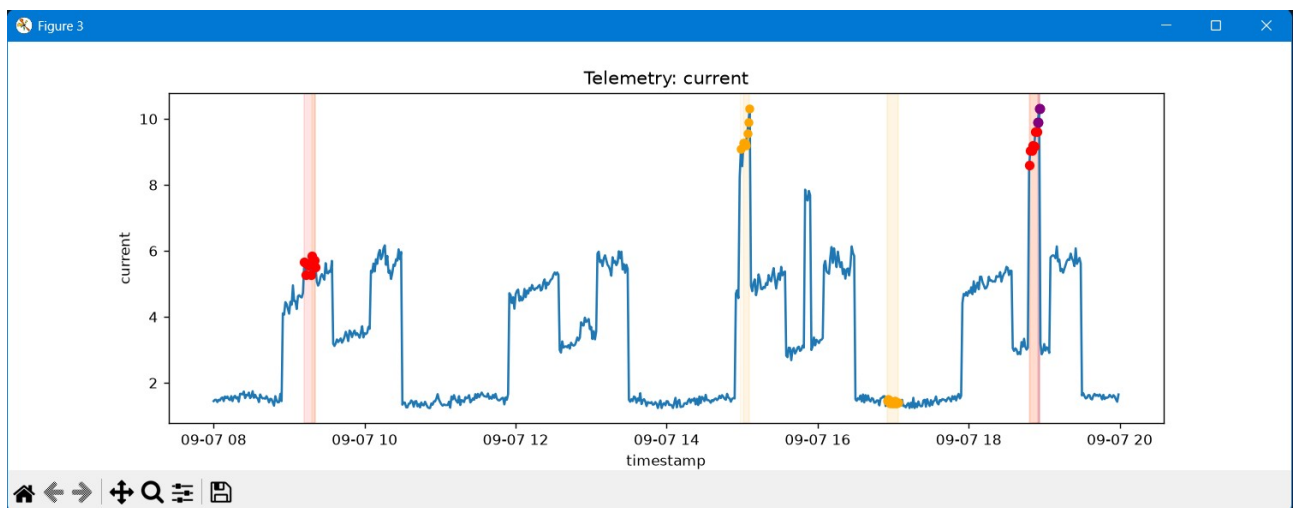


Рисунок 6

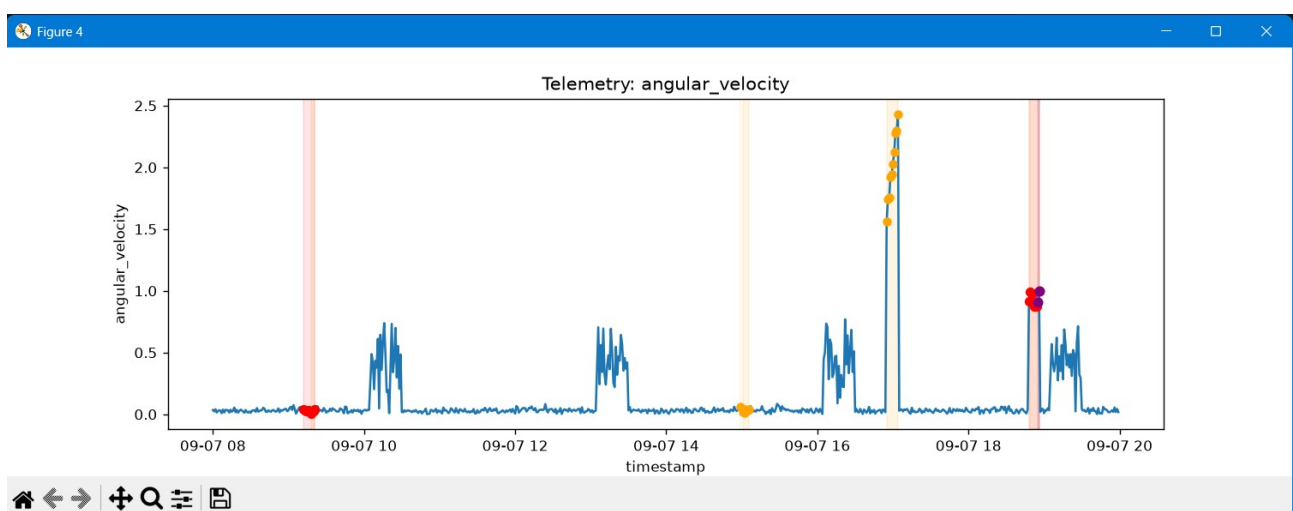


Рисунок 7

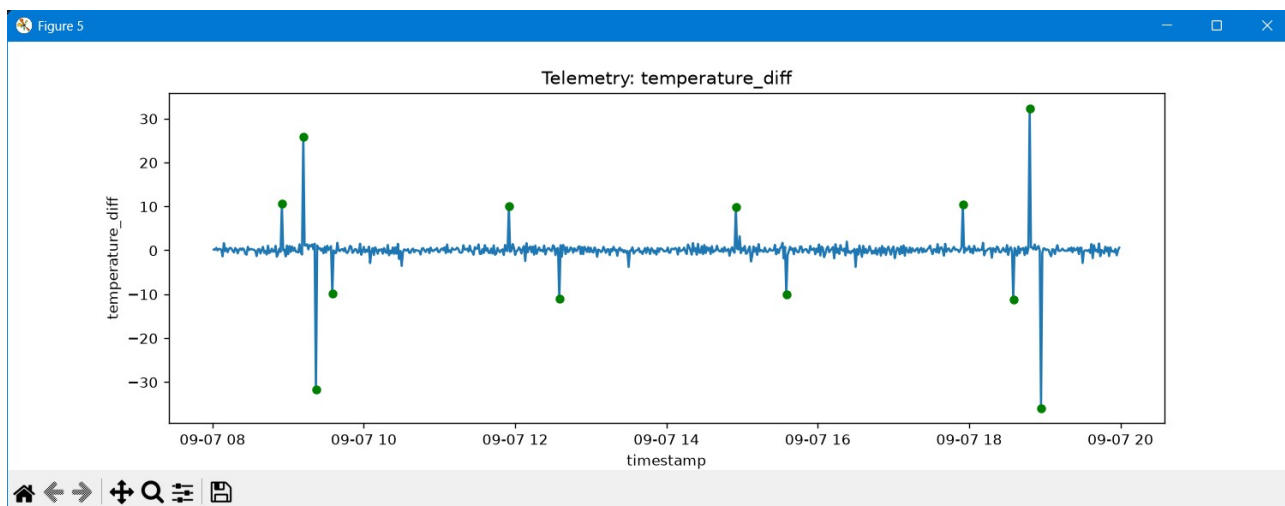


Рисунок 8

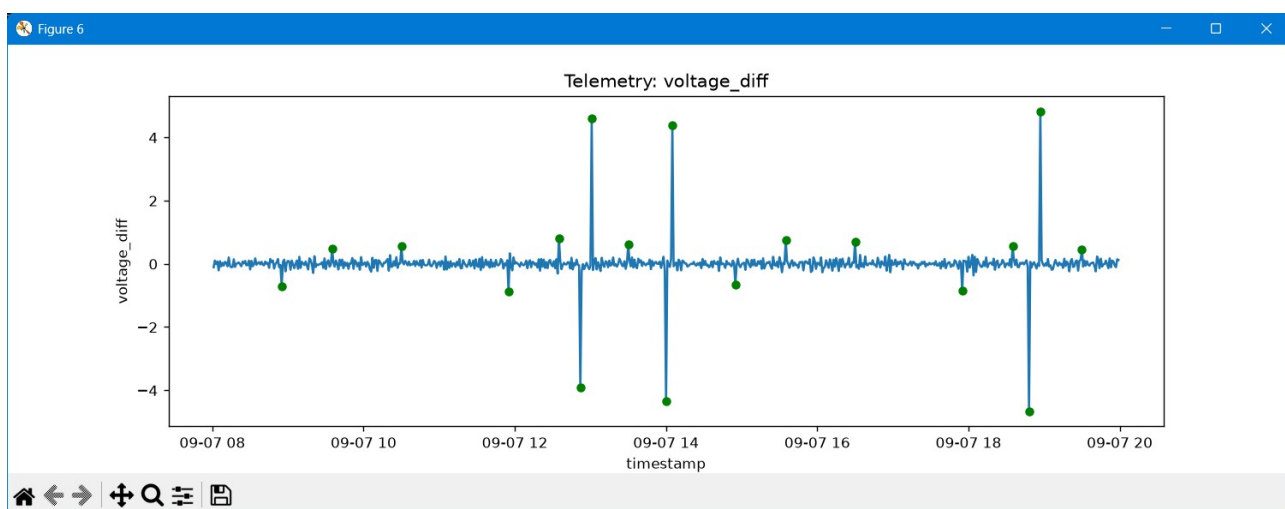


Рисунок 9

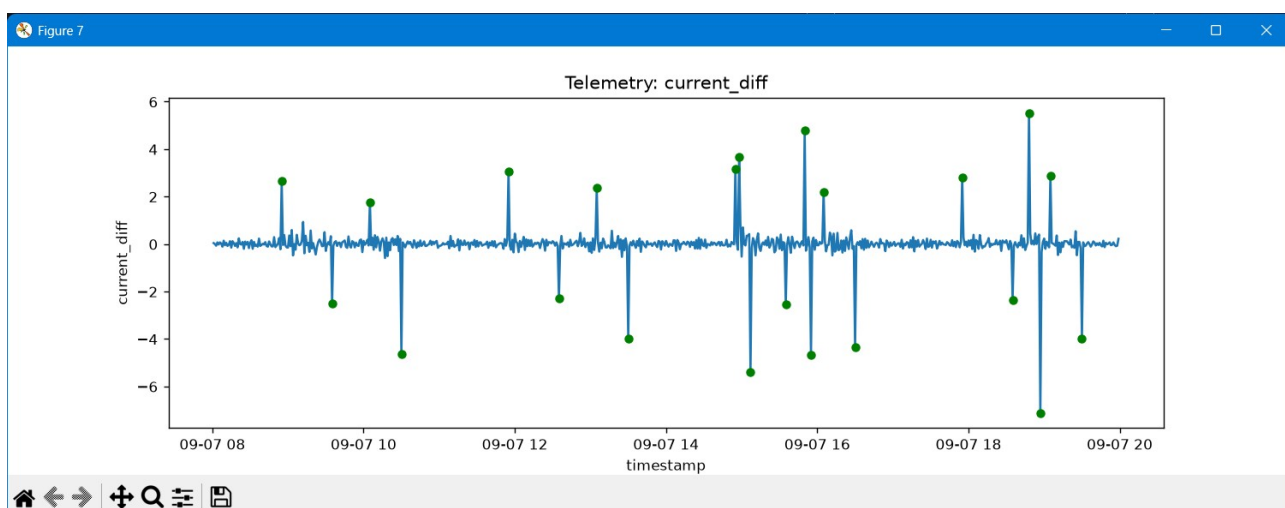


Рисунок 10

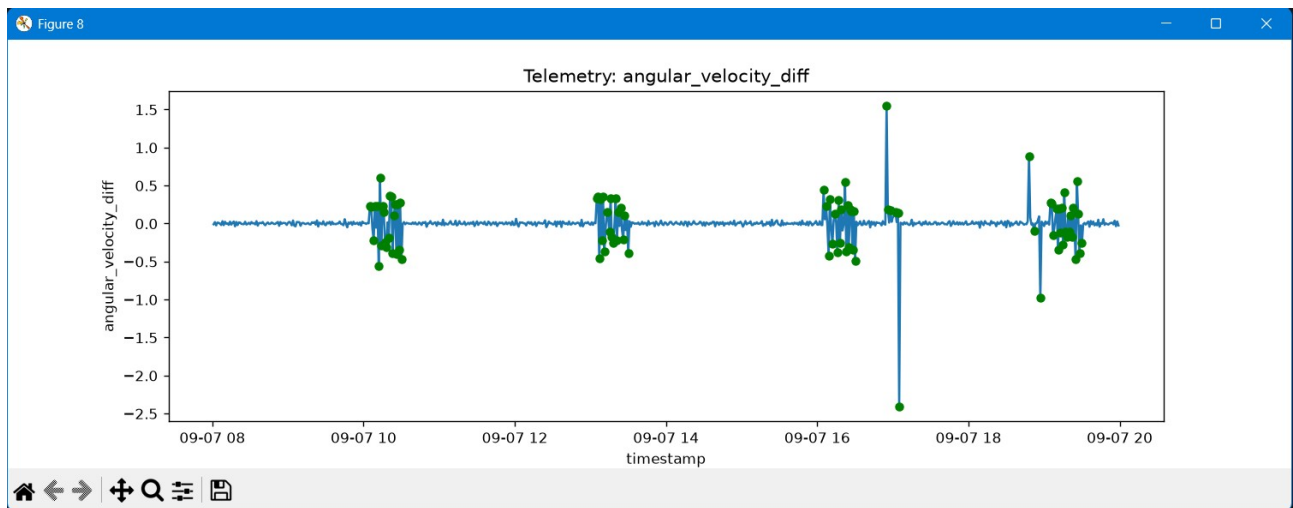


Рисунок 11

ЗАКЛЮЧЕНИЕ

При выполнении задания был доработан простой детектор без машинного обучения. Ссылка на [github](#) с программой.